1. Fused Anisotropic Stencil Filter: Complete Specification

This document provides a self-contained specification of the fused anisotropic stencil filter for oriented edge detection. All equations required to construct and apply the filter are given below, starting from a grayscale image and ending with per-pixel gradient magnitude and edge angle. No prior knowledge of the filter's construction history is assumed.

1.1. Inputs and Parameters

The filter requires a grayscale image and six user-chosen parameters.

$f(X, Y)$	Grayscale image intensity at pixel (X, Y), with $X = 1, ..., N_h$ and $Y = 1, ..., N_v$.
N_s	Number of candidate orientations (e.g. 36 for 5° spacing).
P	Polynomial order for the Taylor expansion (e.g. 4 or 5).
N_p	Number of neighbor pixels in each wide-view neighborhood (e.g. 80–250).
m	Half-length of the line in pixels. The full line contains $2m + 1$ virtual pixels.
σ_w	Standard deviation of the Gaussian weighting along the line.
r	Radius of the circular wide-view neighborhood around each virtual pixel.

The candidate orientations are uniformly spaced over the half-circle.

$$\theta_k = k \cdot \frac{\pi}{N_s}, \quad k = 0, 1, ..., N_s - 1$$

1.2. Step 1: Define the Line of Virtual Pixels

For each orientation θ_k, define $2m + 1$ virtual pixels equally spaced along a line through the target pixel (X_0, Y_0). The j-th virtual pixel sits at global coordinates as follows.

$$X_j^{(k)} = X_0 + j \cos\theta_k, \quad Y_j^{(k)} = Y_0 + j \sin\theta_k, \quad j = -m, ..., 0, ..., m$$

These positions are generally non-integer. Each virtual pixel serves as the center of a local polynomial fit.

1.3. Step 2: Build the Wide-View Neighborhood

Around each virtual pixel $\left(X_j^{(k)}, Y_j^{(k)}\right)$, select the N_p nearest integer-coordinate pixels within radius r. Denote these neighbor pixels as $\left(X_{j,i}, Y_{j,i}\right)$ for $i = 1, ..., N_p$. Compute local offsets relative to the virtual pixel.

$$x_{j,i} = X_{j,i} - X_j^{(k)}, \quad y_{j,i} = Y_{j,i} - Y_j^{(k)}$$

These offsets define the local coordinate system for the polynomial fit at virtual pixel j.

1.4. Step 3: Construct the Taylor Expansion Matrix

At each virtual pixel j, the image intensity at neighbor i is modeled as a 2D Taylor expansion about the virtual pixel location. For polynomial order P, the expansion is written as follows.

$$f(X_{j,i}, Y_{j,i}) \approx \sum_{p=0}^{P} \sum_{q=0}^{P-p} \frac{1}{p! \cdot q!} \cdot \frac{\partial^{p+q} f}{\partial x^p \partial y^q} \cdot x_{j,i}^p \cdot y_{j,i}^q$$

This is assembled into a matrix equation. Each row of the matrix $\boldsymbol{A}_j \in \mathbb{R}^{N_p \times M}$ corresponds to one neighbor pixel, and each column corresponds to one monomial term. The number of monomial terms is $M = \binom{P+2}{2}$. For example, with $P = 4$ the monomials in order are as follows.

$$1, \quad x, \quad y, \quad \frac{x^2}{2}, \quad xy, \quad \frac{y^2}{2}, \quad \frac{x^3}{6}, \quad x^2\frac{y}{2}, \quad x\frac{y^2}{2}, \quad \frac{y^3}{6}, \quad ...$$

The entry in row i, column corresponding to monomial $x^p y^q/(p!q!)$, is simply $x_{j,i}^p \cdot y_{j,i}^q$.

The unknown vector $\boldsymbol{z}_j \in \mathbb{R}^M$ contains the derivative coefficients at virtual pixel j, organized in the same monomial order.

$$\boldsymbol{z}_j = \left[f^0,\ f_x^0,\ f_y^0,\ f_{xx}^0,\ f_{xy}^0,\ f_{yy}^0,\ ...\right]^\top$$

The system of equations is then the following.

$$\boldsymbol{A}_j \boldsymbol{z}_j = \boldsymbol{b}_j$$

where $\boldsymbol{b}_j \in \mathbb{R}^{N_p}$ is the vector of image intensities at the neighbor pixels: $b_{j,i} = f\big(X_{j,i}, Y_{j,i}\big)$.

1.5. Step 4: Solve via Pseudoinverse

Since $N_p > M$ (the system is overdetermined), the least-squares solution is obtained through the Moore–Penrose pseudoinverse.

$$\boldsymbol{z}_j = \boldsymbol{A}_j^+ \boldsymbol{b}_j, \quad \text{where} \quad \boldsymbol{A}_j^+ = \left(\boldsymbol{A}_j^\top \boldsymbol{A}_j\right)^{-1} \boldsymbol{A}_j^\top$$

The pseudoinverse $\boldsymbol{A}_j^+$ is an $M \times N_p$ matrix. Row 2 of $\boldsymbol{A}_j^+$ (0-indexed) extracts f_x^0 and row 3 extracts f_y^0. Denote the row of $\boldsymbol{A}_j^+$ that extracts the normal derivative of interest as $\boldsymbol{a}_j^\top \in \mathbb{R}^{1 \times N_p}$. Then the derivative at virtual pixel j is the following.

$$d_j = \boldsymbol{a}_j^\top \boldsymbol{b}_j = \sum_{i=1}^{N_p} a_{j,i} \cdot f\big(X_{j,i}, Y_{j,i}\big)$$

Which derivative row to select depends on the orientation θ_k. The normal derivative (perpendicular to the line) is a linear combination of f_x^0 and f_y^0. In the rotated local frame of the line, the normal derivative is the following.

$$d_j = -\sin\theta_k \cdot f_x^0 + \cos\theta_k \cdot f_y^0$$

This is obtained by taking the corresponding linear combination of rows 2 and 3 of $\boldsymbol{A}_j^+$.

$$\boldsymbol{a}_j^\top = -\sin\theta_k \cdot \boldsymbol{A}_j^+[2,:] + \cos\theta_k \cdot \boldsymbol{A}_j^+[3,:]$$

1.6. Step 5: Gaussian Weighting Along the Line

The derivative estimates from the $2m + 1$ virtual pixels are combined with Gaussian weights that decay with distance from the line center.

$$w_j = \exp\left(-\frac{j^2}{2\sigma_w^2}\right), \quad j = -m, ..., m$$

The aggregate derivative at the target pixel is the weighted sum of the individual estimates.

$$d = \sum_{j=-m}^{m} w_j \cdot d_j = \sum_{j=-m}^{m} w_j \sum_{i=1}^{N_p} a_{j,i} \cdot f\big(X_{j,i}, Y_{j,i}\big)$$

1.7. Step 6: Fuse into a Single Stencil

The double summation above touches every pixel in the union of all $2m+1$ neighborhoods. Many of these pixel positions overlap between adjacent virtual pixels. Let Ω_k be the set of unique integer-coordinate pixel positions appearing in the union of all neighborhoods for orientation k.

$$\Omega_k = \bigcup_{j=-m}^{m} \big\{ \big(X_{j,i}, Y_{j,i}\big) : i = 1, ..., N_p \big\}$$

Denote the elements of Ω_k as $\left(X_0 + \tilde{\delta}_\ell^x, Y_0 + \tilde{\delta}_\ell^y\right)$ for $\ell = 1, ..., N_k'$, where $N_k' = |\Omega_k|$.

For each unique position ℓ, the fused weight is the sum of all contributions from every virtual pixel j and neighbor index i that map to that position.

$$\alpha_{k,\ell} = \sum_{j,i \text{ such that } (X_{j,i}, Y_{j,i}) = \left(X_0 + \tilde{\delta}_\ell^x, Y_0 + \tilde{\delta}_\ell^y\right)} w_j \cdot a_{j,i}$$

This is the core result. Each $\alpha_{k,\ell}$ is a single scalar that encodes the polynomial fitting, pseudoinverse extraction, derivative selection, and Gaussian line weighting for one pixel position at one orientation.

1.8. Step 7: Apply the Filter

The fused stencil is applied to every pixel in the image as a convolution. For each orientation k, construct a sparse 2D kernel h_k of size $(2S+1) \times (2S+1)$, where S is the maximum stencil radius, by placing the fused weights at their offset positions.

$$h_k\left[\tilde{\delta}_\ell^x + S,\ \tilde{\delta}_\ell^y + S\right] = \alpha_{k,\ell}, \quad \ell = 1, ..., N_k'$$

All other entries of h_k are zero. The filter response at every pixel is then obtained by convolution.

$$R_k(X_0, Y_0) = (h_k \star f)(X_0, Y_0) = \sum_{\ell=1}^{N_k'} \alpha_{k,\ell} \cdot f\Big(X_0 + \tilde{\delta}_\ell^x,\ Y_0 + \tilde{\delta}_\ell^y\Big)$$

Apply all N_s kernels to produce a response volume $G \in \mathbb{R}^{N_s \times N_h \times N_v}$.

$$G_{k,X_0,Y_0} = R_k(X_0, Y_0), \quad k = 0, ..., N_s - 1$$

1.9. Step 8: Extract Gradient Magnitude and Edge Angle

At each pixel, select the orientation with the largest absolute response.

$$k^*(X_0, Y_0) = \arg\max_k |G_{k,X_0,Y_0}|$$

The outputs are the gradient magnitude and the edge angle.

$$\text{magnitude}(X_0, Y_0) = |G_{k^*,X_0,Y_0}|$$

$$\text{angle}(X_0, Y_0) = \theta_{k^*} = k^* \cdot \frac{\pi}{N_s}$$

1.10. Summary of Precomputation

All orientation-dependent quantities ($\boldsymbol{A}_j$, $\boldsymbol{A}_j^+$, $\boldsymbol{a}_j$, w_j, $\alpha_{k,\ell}$, and h_k) depend only on the six parameters and the geometry of the stencil, not on any image data. They are computed once and stored. At runtime, applying the filter to an image of any size requires only the following two operations.

$$G = \mathcal{H} \star f, \quad k^* = \arg\max_k |G_k|$$

Here $\mathcal{H} \in \mathbb{R}^{N_s \times (2S+1) \times (2S+1)}$ is the precomputed filter bank. The first operation is a batched 2D convolution. The second is a pointwise argmax over the orientation axis.